LAPORAN PRAKTIKUM

PEMROGRAMAN BERORIENTASI OBJEK

MODUL 3 – Konsep Dasar Pemrograman Berorientasi Objek



Jury Maxwell

103132400023

PS1DS-05-REG-01

PROGRAM STUDI SAINS DATA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

2026

A. Dasar Teori

Pemrograman Berorientasi Objek (*Object-Oriented Programming*) merupakan metode pembangunan sistem yang komponen-komponentnya di-enkapsulasi menjadi kelompok data dan fungsi, dapat mewarisi sifat dari komponen lain, serta saling berinteraksi satu sama lain. Karakteristik utama dalam pemrograman berorientasi objek meliputi abstraksi, enkapsulasi, pewarisan (*inheritance*), polimorfisme (*polymorphism*), dan pengiriman pesan (*message*).

- Abstraksi berfokus pada penentuan hal-hal esensial dari suatu objek berupa atribut dan perilaku dengan mengabaikan detail yang bersifat insidental.
- Enkapsulasi merupakan proses pemaketan data bersama metode-metodenya yang berfungsi menyembunyikan rincian implementasi (*data hiding*) serta menjaga modularitas kode agar objek dapat dikelola secara independen.
- Pewarisan memungkinkan pembentukan kelas baru (*subclass*) yang mewarisi data dan fungsi dari kelas yang sudah ada (*superclass*).
- Polimorfisme memungkinkan digunakannya satu antarmuka yang sama untuk menghasilkan aksi yang berbeda pada objek yang berbeda. Dalam menjalankan fungsinya, objek-objek saling berkomunikasi dan berinteraksi dengan mengirimkan pesan melalui pemanggilan *method* beserta parameternya.

Dalam struktur pemrograman Java, kelas bertindak sebagai cetak biru (*blueprint*) atau rancangan yang mendefinisikan tipe data dari suatu objek. Suatu kelas memiliki anggota yang terdiri dari atribut untuk menyimpan identitas data dan method sebagai fungsi atau prosedur yang mendefinisikan perilaku dari kelas tersebut. Objek merupakan wujud hasil instansiasi dari suatu kelas. Data di dalam kelas disimpan melalui field, yang dibedakan menjadi *class field* dan *instance field*. *Class field* dideklarasikan menggunakan kata kunci static sehingga nilainya terkait langsung dengan kelas itu sendiri dan diakses melalui nama kelas. Sebaliknya, *instance field* dideklarasikan tanpa kata kunci static, sehingga setiap objek yang diciptakan memiliki salinan datanya sendiri yang bersifat independen dari objek lainnya.

Method digunakan untuk memodularisasi program dengan memisahkan tugas-tugas di dalam kelas, baik dalam bentuk fungsi yang mengembalikan nilai maupun prosedur yang tidak mengembalikan nilai (void). Pengoperan argumen atau parameter ke dalam *method* dilakukan melalui dua mekanisme, yaitu *pass by value* dan *pass by reference*. Mekanisme *pass by value* digunakan untuk tipe data primitif dengan menyalin isi memori, sehingga perubahan variabel di dalam *method* tidak memengaruhi nilai variabel asalnya. Adapun *pass by reference* digunakan pada array dan objek, di mana yang dilewatkan adalah alamat memori data, sehingga setiap perubahan nilai di dalam *method* akan berdampak langsung pada objek atau array asalnya. Selain itu, Java menyediakan kata kunci this sebagai variabel

referensi khusus yang merujuk pada objek yang sedang aktif. Kata kunci this berguna untuk membedakan nama atribut kelas ketika memiliki nama yang sama dengan parameter *method*, serta dapat dimanfaatkan untuk memanggil suatu konstruktor dari konstruktor lainnya di dalam kelas yang sama.

B. Guided

1. Abstraksi

```java
package guided;

public class Kucing {
    String nama;
    String ras;

    void bersuara() {
        System.out.println(nama + " " + ras + " Meow");
    }

}
```

Main:

```java
package guided;

public class Abstraksi {
    public static void main(String[] args) {
        Kucing oren = new Kucing();
        oren.nama = "Cimot Gendot";
        oren.ras = "Angora";
        oren.bersuara();
    }
}
```

Output :

```
PS C:\Users\jmjur\Documents\Co
ceptionMessages' '-cp' 'C:\Use
 'guided.Abstraksi'
Cimot Gendot Angora Meow
```

2. Enkapsulasi

```java
package guided;

public class Rekening {
   private int saldo = 0;
```

```
    public void tambahsaldo (int jumlah) {
        saldo += jumlah;
        System.out.println("Saldo berhasil ditambahkan: " + saldo);
    }

    public void tampilkansaldo(){
        System.out.println("Saldo saat ini: " + saldo);
    }
}
```

Main:

```
package guided;

public class Enkapsulasi {

    public static void main(String[] args) {
        Rekening rek = new Rekening();
        rek.tambahsaldo(300000000);
        rek.tampilkansaldo();
    }
}
```

Output :

```
PS C:\Users\jmjur\Documents\College\Pemrog
ceptionMessages' '-cp' 'C:\Users\jmjur\Doc
 'guided.Enkapsulasi'
Saldo berhasil ditambahkan: 300000000
Saldo saat ini: 300000000
```

3. Field

```
package guided;

public class Circle01 {
    public static final double PI = 3.14;

    public static double radiansToDegrees(double rads) {
        return rads * (180 / PI);
    }

    public double r;

    public double area() {
```

```
        return PI * r * r;
    }

    public double circumference() {
        return 2 * PI * r;
    }

}
```

4. Method

Passed By Reference:

```
package guided;

public class TestPass {
    int i,j;

    TestPass(int a,int b) {
       i = a;
       j = b;
    }
    void calculate(int m,int n) {
        m = m*10;
        n = n/2;
    }
 //passed by reference dengan berupa tipe data class
    void calculate(TestPass e) {
        e.i = e.i*10;
        e.j = e.j/2;
 }
}
```

Passed By Value:

```
package guided;

public class PassedByValue {
    public static void main(String[] args) {
  int x,y;
        TestPass z;
        z = new TestPass(50,100);
        x = 10;
        y = 20;

        System.out.println("Nilai sebelum passed by value : ");
        System.out.println("x = " + x);
```

```
        System.out.println("y = " + y);

        //passed by value
        z.calculate(x,y);
        System.out.println("Nilai sesudah passed by value : ");
        System.out.println("x = " + x);
        System.out.println("y = " + y);
        System.out.println("Nilai sebelum passed by reference : ");
        System.out.println("z.i = " + z.i);
        System.out.println("z.j = " + z.j);

        //passed by reference
        z.calculate(z);

        System.out.println("Nilai sesudah passed by reference : ");
        System.out.println("z.i = " + z.i);
        System.out.println("z.j = " + z.j);
    }
}
```

Output:

```
PS C:\Users\jmjur\Documents\College
ceptionMessages' '-cp' 'C:\Users\jm
 'guided.PassedByValue'
Nilai sebelum passed by value :
x = 10
y = 20
Nilai sesudah passed by value :
x = 10
y = 20
Nilai sebelum passed by reference :
z.i = 50
z.j = 100
Nilai sesudah passed by reference :
z.i = 500
z.j = 50
```

5. Keyword “this”

   Lagu:

```
package guided;
```

```
public class Lagu {
    private String pencipta;
    private String judul;

    public void IsiParam(String judul, String pencipta) {
        this.judul = judul;
        this.pencipta = pencipta;
    }
    public void cetakKeLayar(){
        if(judul==null && pencipta==null) return;
        System.out.println("Judul : " + judul +", pencipta : " +
pencipta);
    }
    static class DemoLagu {
        public static void main(String[] args){
            Lagu a = new Lagu();
            a.IsiParam("God Will Make A Way", "Don Moen");
            a.cetakKeLayar();
        }
    }
}
```

DemoLagu:

```
package guided;

public class DemoLagu {
    public static void main(String[] args) {
        Lagu a = new Lagu();
        a.IsiParam("God Will Make A Way", "Don Moen ");
        a.cetakKeLayar();
    }
}
```

Output:

```
PS C:\Users\jmjur\Documents\College\Pemrograman B
jek\Praktikum'; & 'C:\Users\jmjur\jdk-17\bin\java
mrograman Berbasis Objek\Praktikum\modul-03-konse
Judul : God Will Make A Way, pencipta : Don Moen
```

6. Keyword “this” memanggil sebuah konstruktor dari konstruktor lainnya

```
package guided;

public class Buku {
    private String pengarang;
```

```
    private String judul;
    private Buku() {
        this("Rumah Kita", "GoodBles");
          // this ini digunakan untuk memanggil konstruktor yang
          // menerima dua parameter
    }
    private Buku(String judul,String pengarang)
    {
        this.judul = judul;
        this.pengarang = pengarang;
    }
    private void cetakKeLayar()
    {
        System.out.println("Judul : " + judul + " Pengarang : " +
pengarang);
    }
    public static void main(String[] args)
    {
        Buku a,b ;
        a = new Buku("Jurassic Park", "Michael Chricton");
        b = new Buku();
        a.cetakKeLayar();
        b.cetakKeLayar();
    }
}
```

Output :

```
PS C:\Users\jmjur\Documents\College\Pemrograman Ber
jek\Praktikum'; & 'C:\Users\jmjur\jdk-17\bin\java.e
mrograman Berbasis Objek\Praktikum\modul-03-konsep-
Judul : Jurassic Park Pengarang : Michael Chricton
Judul : Rumah Kita Pengarang : GoodBles
```

C. Unguided

1. Soal Unguided 1

   Sebuah tim mengumpulkan suhu harian suatu kota selama satu minggu (7 hari) dari sensor otomatis. Data ini sudah tersedia sebagai array di main sebelum diserahkan untuk diolah.

   Pada salah satu hari, sensor gagal mengirim data. Hari yang gagal terekam ditandai dengan sebuah nilai penanda pada array, karena suhu asli tidak mungkin sebesar itu.

Nilai penanda ini sama untuk semua data suhu yang diolah kapan pun dan oleh siapa pun bukan sesuatu yang berbeda-beda tiap kali data dibuat sehingga sebaiknya ditulis satu kali saja sebagai aturan tetap milik class, bukan diketik ulang sebagai angka -1.0 di setiap method yang membutuhkannya.

Buatlah satu class yang bertugas mengolah data suhu ini memeriksa apakah ada data kosong, mengisinya, dan menghasilkan statistik dasar dari data yang sudah bersih.
Jawaban :

PengolahSuhu:

```
package unguided;

public class PengolahSuhu {
    public static final double NILAI_KOSONG = -1.0;
    private double[] suhuHarian;

    public PengolahSuhu(double[] suhuHarian) {
        this.suhuHarian = suhuHarian;
    }

    public void tampilkanData() {
        for (int i = 0; i < suhuHarian.length; i++) {
            if (suhuHarian[i] == NILAI_KOSONG) {
                System.out.println("Hari " + (i + 1) + " :
(kosong)");
            } else {
                System.out.println("Hari " + (i + 1) + " : " +
suhuHarian[i] + "°C");
            }
        }
    }

    public int cariIndexKosong() {
        for (int i = 0; i < suhuHarian.length; i++) {
            if (suhuHarian[i] == NILAI_KOSONG) {
                return i;
            }
        }
        return -1;
    }

    public void isiDataKosong() {
        int i = cariIndexKosong();
        if (i != -1) {
```

```
            suhuHarian[i] = (suhuHarian[i - 1] + suhuHarian[i + 1]) /
2;
        }
    }

    public double hitungRataRata() {
        double total = 0;
        for (int i = 0; i < suhuHarian.length; i++) {
            total += suhuHarian[i];
        }
        return total / suhuHarian.length;
    }
}
```

Main:

```
package unguided;

public class Main {
    public static void main(String[] args) {
        double[] suhuHarian = { 30.4, 24.3, 26.8, -1.0, 31.4, 30.8,
32.9 };

        PengolahSuhu pengolah = new PengolahSuhu(suhuHarian);

        System.out.println("=== Data Suhu Awal ===");
        pengolah.tampilkanData();

        System.out.println("\nIndex hari kosong (dimulai dari 0): " +
pengolah.cariIndexKosong());

        pengolah.isiDataKosong();

        System.out.println("\n=== Data Suhu Setelah Pengisian ===");
        pengolah.tampilkanData();

        double rataRata = Math.round(pengolah.hitungRataRata() *
100.0) / 100.0;
        System.out.println("\nRata-rata : " + rataRata + "°C\n");

        System.out.println("Isi array suhuHarian di main setelah
isiDataKosong() dijalankan:");
        System.out.print("[");
        for (int i = 0; i < suhuHarian.length; i++) {
            System.out.print(suhuHarian[i]);
            if (i < suhuHarian.length - 1) {
```

```
                System.out.print(", ");
            }
        }
        System.out.println("]");
        System.out.println("(ikut berubah: constructor menyimpan
referensi array yang sama)");
    }
}
```

Output :

```
PS C:\Users\jmjur\Documents\College\Pemrograman Berbasis Objek\Pr
jek\Praktikum'; & 'C:\Users\jmjur\jdk-17\bin\java.exe' '-XX:+Shou
mrograman Berbasis Objek\Praktikum\modul-03-konsep-dasar\project'
=== Data Suhu Awal ===
Hari 1 : 30.4°C
Hari 2 : 24.3°C
Hari 3 : 26.8°C
Hari 4 : (kosong)
Hari 5 : 31.4°C
Hari 6 : 30.8°C
Hari 7 : 32.9°C

Index hari kosong (dimulai dari 0): 3

=== Data Suhu Setelah Pengisian ===
Hari 1 : 30.4°C
Hari 2 : 24.3°C
Hari 3 : 26.8°C
Hari 4 : 29.1°C
Hari 5 : 31.4°C
Hari 6 : 30.8°C
Hari 7 : 32.9°C

Rata-rata : 29.39°C

Isi array suhuHarian di main setelah isiDataKosong() dijalankan:
[30.4, 24.3, 26.8, 29.1, 31.4, 30.8, 32.9]
(ikut berubah: constructor menyimpan referensi array yang sama)
```

Penjelasan :

Program ini berfungsi untuk mengolah dan mengisi data suhu mingguan yang tidak lengkap. Alur kerja program dimulai paada kelas Main dengan menyiapkan array suhu mentah yang kemudian diteruskan ke objek PengolahSuhu melalui constructor dengan menggunakan keyword this. Objek lalu menampilkan representasi data awal, sekaligus mendeteksi posisi index hari yang kehilangan data dengan mencocokan angka konstanta -1.0. Pengisian nilai null dilakukan melalui metode imputasi deret waktu, dengan menghitung rata-rata dari dua titik data tetangga terdekatnya (satu hari sebelum dan satu hari sesudah) index yang kosong. Setelah array terisi secara utuh, program menampilkan data suhu akhir yang telah bersih serta menghitung rata-rata mingguannya. Eksekusi diakhiri dengan mencetak kembali variable array pada kelas Main untuk menunjukkan data telah diperbarui, membuktikan mekanisme pass by reference dimana objek dan kelas utama saling berbagi lokasi memori yang sama.

D. Kesimpulan

Pada praktikum modul 3 ini saya mempelajari apa saja karakteristik OOP seperti Abstraksi, Enkapsulasi, Inheritance dan Polymorpisms. Praktikum kali ini juga menjelaskan bagian bagian yang ada dalam OOP seperti Kelas, Object, Field beserta jenisnya, Method dan juga Keyword “this”.